

Режимы Мыйзамчы — полная карта (расширенная)

Полная техническая карта всех режимов ИИ в проекте Мыйзамчы (`server.js`): агенты, топ-K, пороги score, число вызовов, модели, параметры генерации, ретраи.

Актуально на: 2026-05-22. Источник истины — `server.js` . Все числа взяты из констант кода, не приблизительные.

0. Краткая карта (TL;DR)

Группа	Режим	Эндпоинт	Pinecone topK	LLM-вызовов*	Макс. агентов
A	Болталка	<code>/api/chat</code> <code>fast</code>	— (поиск выключен)	1	1
A	Быстрый	<code>/api/chat</code> <code>fast</code>	10	2	1
A	Думающий → Простой	<code>/api/chat</code> <code>thinking</code>	15	2–3	2
A	Думающий → Глубокий	<code>/api/chat</code> <code>thinking</code>	5+10+8+6+5 = 34	7–8	7
Б	Агент-редактор	<code>/api/chat</code> <code>agentMode</code>	10 / 15 / 22 (адаптив)	2	1
Б	AI-редактор	<code>/api/edit</code>	—	1	1
В	Проверка документа	<code>/api/analyze-document</code>	5 на статью/пункт	~25–35	до 56
В	Глубокий анализ (PRO)	<code>/api/deep-analyze-document</code>	5	~12–20	17 LLM + до 30 проверок
Г	Сравнение редакций	<code>/api/compare-documents</code>	— (cos-выравнивание)	~104 (1 конт + 2 сегм + 100 ворк + 1 DS)	до 104

* «LLM-вызов» = запрос к генеративной модели. Эмбединги и Pinecone — отдельно.

1. Инфраструктура (общая для всех режимов)

1.1 Модели

Назначение	Модель	Константа
Основная (PRIMARY)	<code>gemini-2.5-flash</code> (Google, stable)	<code>PRIMARY_MODEL</code>
Резервная (FALLBACK)	<code>gemini-2.5-flash-lite</code> (Google)	<code>FALLBACK_MODEL</code>
Эмбединги	<code>gemini-embedding-001</code> , 768 измерений	<code>EMBEDDING_MODEL</code>
Судья / Старший партнёр	<code>deepseek-v4-pro</code> (DeepSeek)	<code>DEEPSEEK_MODEL</code>

DeepSeek V4 Pro: OpenAI-совместимый SDK, `baseUrl: https://api.deepseek.com` , `reasoning_effort: 'high'` , `thinking: {type:'enabled'}` . Мягкая зависимость — если ключа нет или DeepSeek упал, автоматический откат на Gemini.

1.2 RAG — профили `adaptiveRetrieval`

`adaptiveRetrieval(query, mode, res, opts)` шлёт в Pinecone `topK = maxK` , затем фильтрует elbow-методом до `minK..maxK` статей.

Профиль	maxK (= Pinecone topK)	minK (минимум после фильтра)
<code>fast</code>	10	3
<code>agent</code>	15	4
<code>thinking</code>	25	5

`opts.maxK` / `opts.minK` переопределяют дефолт профиля — многие режимы так и делают.

1.3 Пороги отбора статей (score)

Параметр	Значение	Смысл
<code>absoluteMinScore</code>	0.45	ниже — статья отбрасывается полностью
<code>coreScoreThreshold</code>	0.75	$\geq 0.75 \rightarrow \star$ ключевая, ниже $\rightarrow \text{📖}$ вспомогательная
<code>elbowDropRatio</code>	0.15	обрыв ранжирования: падение score > 15% $\rightarrow$ отсечка
<code>confidenceThresholds</code>	high 0.75 · medium 0.70 · low 0.60	уверенность сверки статьи в анализе документа

1.4 Параметры генерации

Вызов	temperature	maxOutputTokens	стриминг
Болталка / Быстрый (<code>handleFast</code>)	по умолчанию (~1.0)	по умолчанию	да

Вызов	temperature	maxOutputTokens	стриминг
Простой / Глубокий синтез (<code>streamGeminiResponse</code>)	по умолчанию	по умолчанию	да
Агент-редактор (<code>handleAgent</code>)	0.4 · topP 0.9	4096	да
AI-редактор (<code>/api/edit</code>)	по умолчанию	по умолчанию	нет
Судья — анализ документа (<code>runJudge</code>)	0.2	4096 (при откате на Gemini)	да
Старший партнёр (<code>seniorPartnerSynthesis</code>)	0.25	2048 (при откате)	нет
Драфтер (<code>drafterGenerate</code>)	0.35 · topP 0.9	3500	нет
Воркеры (<code>callOnce</code>)	по умолчанию	по умолчанию	нет

1.5 Надёжность и ретрай

Механизм	Где	Поведение
<code>generateContentResilient</code>	все воркеры через <code>callOnce</code>	4 попытки (maxRetries=3), backoff <code>800мс · 2ⁿ</code> + jitter 0–800мс, потолок 8с. Последняя попытка → FALLBACK_MODEL . На 404 — сразу на резерв.
<code>handleFast</code> собственный ретрай	Болталка, Быстрый	до <code>KEYS.length</code> повторов, пауза 800мс, модель не меняется (всегда PRIMARY)
<code>handleAgent</code> собственный ретрай	Агент-редактор	до <code>min(KEYS.length, 3)</code> , пауза 1500мс
<code>deepseekCall</code>	Старший партнёр	3 попытки , backoff потолок 4с, затем откат на Gemini
<code>streamDeepSeekResponse</code>	Судья	1 попытка; сбой до 1-го чанка → откат на Gemini-стрим
Блокировка ключа (<code>blockKey</code>)	ротация	15 сек , ТОЛЬКО при 429 (не при 503)
Ротация ключей	Gemini	round-robin по <code>GEMINI_API_KEY</code> (через запятую)
Rate limit <code>/api/chat</code>	—	30 запросов / мин
Rate limit <code>/api/analyze-document</code>	—	30 запросов / мин
Rate limit <code>/api/deep-analyze-document</code>	—	6 запросов / мин
Rate limit <code>/api/compare-documents</code>	—	8 запросов / мин

1.6 Лимиты контекста и объёмов

Параметр	Значение
История чата (<code>sanitizeHistory</code>)	последние 10 сообщений
Текст для эмбединга (<code>getEmbedding</code>)	обрезка до 8000 символов
Кэш эмбедингов	200 записей (LRU)
Кэш прокси Минюста	200 записей, TTL 1 час
Запрос тела API	лимит 2 МБ
Мин. длина документа для анализа	50 символов
Макс. статей из документа (<code>maxArticles</code>)	30
Макс. пунктов документа (<code>SEGMENT_LIMIT</code>)	25
Макс. пунктов для сравнения (<code>COMPARE_SEG_LIMIT</code>)	60
Чанк экстрактора статей	7000 симв., overlap 500
Чанк сегментатора	7500 симв., overlap 400
Голова для «паспорта» документа	первые 4500 симв.
Документ в промпт Судьи	обрезка до 15000 симв.
Запрос эмбединга при проверке пункта	обрезка до 450 симв.

ГРУППА А — ЧАТ (вкладка «Чат»)

Тело запроса: `{ message, history, mode }, mode = fast | thinking`.

A.0 — Болталка (приветствие)

Параметр	Значение
Триггер	<code>isCasualMessage()</code> — регех по <code>GREETING_PATTERNS</code> , длина ≤ 60 символов
Эндпоинт	<code>/api/chat</code> , <code>mode: fast</code>
Агентов	1
Pinecone topK	— (поиск выключен)
Эмбедингов	0
LLM-вызовов	1 (стриминг)
Модель	<code>gemini-2.5-flash</code> , ретрай без смены модели
Промпт	<code>systemInstruction</code> (раздел «Режим 0»)

Параметр	Значение
Контекст	только история (10 сообщений)

A.1 — Быстрый режим (fast)

Параметр	Значение
Триггер	<code>mode: fast</code> , сообщение не болталка
Агентов	1
Pinecone topK	10 (профиль <code>fast</code> , minK 3)
Эмбедингов	1 (+ кэш)
LLM-вызовов	1 (стриминг)
Модель	<code>gemini-2.5-flash</code>
Промпт	<code>systemInstruction</code> (+ <code>L4_WARNING_ADDON</code> при просьбе составить иск)
Контекст	до 10 статей, ★ ≥ 0.75 / 📄 < 0.75
Ретрай	свой, до <code>KEYS.length</code> , без смены модели

A.2 — Думающий режим (thinking)

Сначала классификатор `classifyQuery()` : `quickClassify()` (regex) → если неясно, `llmClassify()` (1 LLM-вызов). Далее один из двух подрежимов.

A.2a — Простой запрос (`handleSimpleConsultation`)

Параметр	Значение
Триггер	классификатор → <code>simple</code> (термин, «какая статья», цифра-факт)
Агентов	2 (классификатор + синтезатор)
Pinecone topK	15 (профиль <code>thinking</code> с <code>opts.maxK=15</code> , minK 4)
Эмбедингов	1
LLM-вызовов	2–3 (0–1 классификатор + 1 синтез)
Модель	<code>gemini-2.5-flash</code>
Промпт	<code>BASE_CONSULTANT_PROMPT</code> (+ <code>ACADEMIC_PROMPT_ADDON</code> / <code>L4_WARNING_ADDON</code>)
Ретрай синтеза	1 повтор (на запасном промпте <code>systemInstruction</code>)

A.2b — Глубокий запрос (`handleDeepThinking`) — 5 слоёв

Параметр	Значение
Триггер	классификатор → <code>complex</code> (реальная ситуация, действия, документ)
Агентов	7 (реформулятор + 5 слоёв поиска + синтезатор)
Эмбедингов	5
LLM-вызовов	7–8 (0–1 классиф. + 1 реформулировка + 1 синтез)
Модель	<code>gemini-2.5-flash</code>
Промпт	<code>BASE_CONSULTANT_PROMPT</code> + блок «Иерархический контекст (5 слоёв)»

5 параллельных слоёв поиска (топ-K у каждого свой):

Слой	Профиль	Pinecone topK	minK
★ Специальная норма	<code>fast</code>	5	3
🏛️ Общие положения	<code>thinking</code>	10	4
👮 Процессуальные нормы	<code>thinking</code>	8	3
⚡ Ответственность	<code>fast</code>	6	2
📄 Подзаконные акты	<code>fast</code>	5	2
Итого		34	

ГРУППА Б — АГЕНТ (вкладка «Агент»)

Б.1 — Агент-редактор (Cursor-style)

Роутер: документ < 100 символов → запрос уходит в А.2 (Простой/Глубокий).

Параметр	Значение
Триггер	<code>agentMode: true</code> + открытый документ ≥ 100 символов
Агентов	1
Pinecone topK	адаптивный: запрос >200 симв. → 22 · >60 → 15 · иначе → 10
minK	6 / 4 / 3 соответственно
Эмбедингов	1

Параметр	Значение
LLM-вызовов	1 (стриминг)
Модель	gemini-2.5-flash , temperature 0.4 , topP 0.9 , maxOutputTokens 4096
Промпт	AGENT_SYSTEM_PROMPT (строгий JSON {reasoning, anchor_text, insertion_text})
Ретрай	свой, до min(KEYS.length, 3) , пауза 1500мс

Б.2 — AI-редактор (/api/edit)

Параметр	Значение
Триггер	действие «переписать выделенное» в IDE
Агентов	1
Pinecone topK	— (поиска нет)
LLM-вызовов	1 (НЕ стриминг, generateContent)
Модель	gemini-2.5-flash
Промпт	inline: «юрист-редактор КР, верни только исправленный текст»
Rate limit	отдельного нет (общий не применяется к /api/edit)

ГРУППА В — АНАЛИЗ ДОКУМЕНТА

В.1 — Проверка документа (/api/analyze-document)

Оркестратор analyzeDocumentSmart() . Гибрид: проверка статей + проверка пунктов.

Этап	Агент	Pinecone topK	LLM
0. Паспорт документа	extractDocumentContext	—	1
1. Извлечь статьи	extractArticles	—	1 + по 1 на чанк (7000/500)
1. Разбить на пункты	segmentDocument	—	1 + по 1 на чанк (7500/400)
2. Проверка статей	verifySingleArticle ×N	5 на статью	0 (regex-сверка)
2. Проверка пунктов	verifySegment ×N	5 на пункт	1 на пункт
4. Заключение	runJudge	—	1 DeepSeek V4 Pro

Лимиты-агенты:

Что	Макс.
Верификаторы статей (параллельно)	30 (<code>maxArticles</code>) — без LLM
Верификаторы пунктов (параллельно)	25 (<code>SEGMENT_LIMIT</code>) — каждый с LLM
Судья	1
Итого макс. параллельных агентов	до 56 (30 + 25 + 1)

Пример (12 статей + 25 пунктов, скриншот «30/41»): ~4–6 LLM
(паспорт+извлечение+сегментация) + **25 LLM-вердиктов по пунктам** + 37 эмбедингов + 37 Pinecone (topK 5) + 1 DeepSeek ≈ **30 LLM Gemini + 1 DeepSeek**.

В.2 — Глубокий анализ PRO (`/api/deep-analyze-document`)

Оркестратор `runDeepAnalysis()` . Архитектура «Старший партнёр + команда».
Параметры запроса: `perspective` (`ours` / `opponent` / `audit`) · `modules` (любая комбинация `audit` , `strategy` , `drafter` , `mentor` — **макс. 4**).

Агент / модуль	Под-агенты	Pinecone topK	LLM-вызовов
Паспорт + сегментация	—	—	2 (+ чанки)
Аудитор (<code>audit</code>)	<code>redFlags</code> · <code>collisions</code> · <code>procKiller</code> · <code>factCheck</code>	<code>factCheck</code> : 5 на статью	3 + 1 (экстрактор внутри <code>factCheck</code>)
↳ <code>factCheck</code> → верификаторы статей	до 30	5	0 (regex)
Стратег (<code>strategy</code>)	<code>heatmap</code> + контраргументы	5 на контраргумент	1 + до 6
Драфтер (<code>drafter</code>)	—	—	1
Ментор (<code>mentor</code>)	<code>opponentSim</code> · <code>judgeSim</code>	—	2
Старший партнёр	—	—	1 DeepSeek V4 Pro

Максимум агентов за один прогон (все 4 модуля включены):

```
контекст(1) + сегментатор(1)
+ Аудитор: redFlags(1) + collisions(1) + procKiller(1) + factCheck-экстрактор(1)      = 4
+ Стратег: heatmap(1) + контраргументы(до 6)                                     = 7
+ Драфтер(1)
+ Ментор: opponentSim(1) + judgeSim(1)                                           = 2
+ Старший партнёр(1, DeepSeek)
= 17 LLM-агентов + до 30 верификаторов статей (без LLM, внутри factCheck)
```

Жёсткие лимиты: контраргументов — **6** (`threats.slice(0,6)`), `red flags` / коллизий / проц.дефектов — по 6–8, тепловая карта — 25 пунктов, атак ментора — 5, вопросов

судьи — 5.

Порядок: Аудитор II Стратег → (Драфтер II Ментор) → Старший партнёр.

ГРУППА Г — СРАВНЕНИЕ РЕДАКЦИЙ (Semantic Legal Redlining)

Новый режим в IDE (вкладка «**Сравнение**» в pill «Чат · Агент · Сравнение»). Юрист загружает две версии документа — система показывает **семантические** изменения (не диффы букв), оценивает риски и выдаёт Executive Summary.

Г.1 — Сравнение двух редакций (/api/compare-documents)

Файл: `routes/compare.js` (единственный режим, вынесенный в отдельный модуль — фабрика, которой `server.js` передаёт хелперы; циркулярного require нет).

Архитектура Map-Reduce — четыре этапа:

Этап	Что делает	Pinecone	LLM
0. CONTEXT	<code>extractDocumentContext</code> извлекает тип, отрасль и стороны договора (инжектируется в сегментатор и воркеры)	—	1
1. ALIGN	сегментация обоих документов с лимитом <code>COMPARE_SEG_LIMIT = 60</code> + эмбединги каждого пункта + жадное сопоставление по cosine similarity (порог ≥ 0.78)	— (используются только эмбединги)	2–4 (сегментаторы)
2. MAP	пары → 1 пара = 1 агент (<code>WORKER_BATCH_SIZE = 1</code>) → параллельные воркеры на <code>gemini-2.5-flash</code> (<code>temperature: 0.2</code> , <code>maxOutputTokens: 2048</code>) с учетом контекста документа дают вердикт: <code>hasChanges</code> / <code>category</code> / <code>riskDetected</code> / <code>riskDescription</code>	—	до ~100 воркеров (волнами по <code>WORKER_CONCURRENCY = 12</code>)
3. REDUCE	фильтр важных изменений → компактная дельта (НЕ полные тексты, чтобы уложиться в окно судьи 12K) → DeepSeek V4 Pro (с <code>reasoning_effort: 'medium'</code>) стримит Executive Summary	—	1 DeepSeek (с откатом на Gemini)

Параметры:

Параметр	Значение	Где задано
Порог выравнивания cos-similarity	0.78	<code>ALIGN_THRESHOLD</code>
Размер батча воркера	1 пара	<code>WORKER_BATCH_SIZE</code>
Макс. пар в обработке	100	<code>MAX_PAIRS</code>
Макс. пунктов на документ	60	<code>COMPARE_SEG_LIMIT</code> (отдельный)
Мин. длина каждого документа	50 симв.	<code>MIN_DOC_LEN</code>
Лимит одновременных LLM-воркеров	12	<code>WORKER_CONCURRENCY</code>
Лимит одновременных эмбедингов	20	<code>EMBED_CONCURRENCY</code>
Запрос для эмбединга пункта	до 1500 симв.	<code>EMBED_QUERY_MAX</code>
Текст пункта в промпте воркера	до 1200 симв.	<code>SEGMENT_TEXT_LIMIT</code>
Дельта для судьи	до 12000 симв.	<code>DELTA_CHARS_LIMIT</code>
temperature воркеров	0.2	inline в <code>runWorkerBatch</code>
temperature судьи	0.25	inline в вызове <code>streamDeepSeekResponse</code>
reasoning судьи (DeepSeek)	medium	inline в вызове <code>streamDeepSeekResponse</code>
Rate limit	8 / мин	отдельный <code>compareLimiter</code>

Категории изменений (выдаёт воркер для каждой пары):

Категория	Значение
<code>стилистика</code>	формулировки переписаны, юр. смысл не изменился
<code>существенное изменение</code>	права/обязанности/сроки/суммы реально изменены
<code>добавление</code>	пункт есть только в новой редакции
<code>удаление</code>	пункт был в старой, в новой отсутствует

`riskDetected: true` ставится ТОЛЬКО когда новая редакция объективно хуже для нашей стороны (выше ответственность, сокращены права, иная подсудность и т.п.).

Максимум агентов за один прогон:

```
1 экстрактор контекста
+ 2 сегментатора (старый || новый)
+ до 200 эмбедингов (100 + 100) – без LLM
+ до 100 воркеров-аудиторов (1 пара = 1 агент) – параллельно (волнами по 12)
+ 1 Старший партнёр (DeepSeek V4 Pro) – стрим финального резюме (reasoning: medium)
= до 104 LLM-агентов + 200 эмбедингов
```

SSE-события клиенту (фронт `<CompareMode/>` уже умеет их рендерить):

Событие	Когда
<code>{ step: { id, status, text } }</code>	для каждого этапа: segment / embed / align / map / judge
<code>{ protocolStatus }</code>	статусные строки для тоста
<code>{ compareReport: { total, matched, added, removed, risksCount, substantialCount, pairs[] } }</code>	один раз — ПОЛНЫЙ side-by-side отчёт ДО синтеза, фронт сразу рисует
<code>{ text }</code>	чанки Executive Summary от DeepSeek
<code>[DONE]</code>	конец

Промпты: `WORKER_SYSTEM_PROMPT` , `JUDGE_SYSTEM_PROMPT_COMPARE` — оба inline внутри `routes/compare.js` , у обоих явный запрет на упоминание внутренней кухни системы.

Фронтенд: компонент `<CompareMode/>` в IDE — две текст-зоны (или `.txt` -загрузка) → кнопка «Сравнить» → стриминговый степпер + Executive Summary сверху + двухколоночный side-by-side viewer с цветной подсветкой (● риск · ● сущ. изменение · ● добавлено · ● удалено) и кликом на пункт для всплывающей карточки с описанием риска.

2. Сводка вызовов

Режим	Эмбединги	Pinecone (topK)	LLM Gemini	DeepSeek
Болталка	0	0	1	0
Быстрый	1	1 × topK10	1	0
Думающий → Простой	1	1 × topK15	1–2	0
Думающий → Глубокий	5	5 (topK 5/10/8/6/5)	2–3	0
Агент-редактор	1	1 × topK10–22	1	0
AI-редактор	0	0	1	0
Проверка документа	N статей + N пунктов	каждый × topK5	~3 + N пунктов	1
Глубокий анализ PRO	~6–36	каждый × topK5	~12–18	1
Сравнение редакций	до 200 (100+100)	—	до 103 (1 конт. + 2 сегм. + до 100 ворк.)	1 (reasoning: medium)

3. Справочник промптов

Константа	Режим
<code>systemInstruction</code>	Болталка, Быстрый
<code>BASE_CONSULTANT_PROMPT</code>	Думающий (Простой + Глубокий)
<code>ACADEMIC_PROMPT_ADDON</code>	+ при запросе курсовой / CPC / реферата
<code>L4_WARNING_ADDON</code>	+ при просьбе составить иск / жалобу в суд
<code>AGENT_SYSTEM_PROMPT</code>	Агент-редактор
<code>JUDGE_SYSTEM_PROMPT</code>	Финал «Проверки документа»
<code>SENIOR_PARTNER_PROMPT</code>	Финал «Глубокого анализа PRO»
<code>WORKER_SYSTEM_PROMPT</code> (compare.js)	Воркер-аудитор пар в «Сравнении»
<code>JUDGE_SYSTEM_PROMPT_COMPARE</code> (compare.js)	Старший партнёр — Executive Summary в «Сравнении»

Промпты воркеров (экстрактор, сегментатор, посегментный судья, Аудитор, Стратег, Драфтер, Ментор) заданы inline внутри соответствующих функций.

4. Что НЕ относится к режимам IDE

- **Telegram-бот** (`./telegram/bot`) — отдельный канал.
- `/api/minjust/*` — прокси к cbd.minjust.gov.kg с кэшем (200 записей, TTL 1ч).
- `/api/stats` , `/health` , `/ping` — служебные эндпоинты.

Сгенерировано по коду `server.js` на 2026-05-22. При изменении моделей, топК, лимитов или промптов — обновить этот файл.